

Copyright 2025 Google LLC.

```
#title Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
# https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
```

 View on al.google.dev  Run in Google Colab  Run in Kaggle  Open in Vertex AI  View source on GitHub

Generate Embeddings with Sentence Transformers

EmbeddingGemma is a lightweight, open embedding model designed for fast, high-quality retrieval on everyday devices like mobile phones. At only 308 million parameters, it's efficient enough to run advanced AI techniques, such as Retrieval Augmented Generation (RAG), directly on your local machine with no internet connection required.

Setup

Before starting this tutorial, complete the following steps:

- Get access to Gemma by logging into [Hugging Face](#) and selecting **Acknowledge license** for a Gemma model.
- Generate a Hugging Face [Access Token](#) and use it to login from [Access Token](#).

This notebook will run on either CPU or GPU.

Install Python packages

Install the libraries required for running the EmbeddingGemma model and generating embeddings. Sentence Transformers is a Python framework for text and image embeddings. For more information, see the [Sentence Transformers](#) documentation.

```
In [ ]: !pip install -U sentence-transformers git+https://github.com/huggingface/transformers@v4.56.0-Embedding-Gemma-preview
```

After you have accepted the license, you need a valid Hugging Face Token to access the model.

```
In [ ]: # Login into Hugging Face Hub
from huggingface_hub import login
login()
```

Load Model

Use the `sentence-transformers` libraries to create an instance of a model class with EmbeddingGemma.

```
In [ ]: import torch
from sentence_transformers import SentenceTransformer

device = "cuda" if torch.cuda.is_available() else "cpu"

model_id = "google/embeddinggemma-300M"
model = SentenceTransformer(model_id).to(device=device)

print(f"Device: {model.device}")
print(model)
print(f"Total number of parameters in the model:", sum([p.numel() for _, p in model.named_parameters()])))

Device: cuda:0
SentenceTransformer(
  (0): Transformer({'max_seq_length': 2048, 'do_lower_case': False, 'architecture': 'Gemma3TextModel'})
    (1): Pooling({'word_embedding_dimension': 768, 'pooling_mode_cls_token': False, 'pooling_mode_mean_tokens': True, 'pooling_mode_max_tokens': False, 'pooling_mode_mean_sqrt_len_tokens': False, 'pooling_mode_weightedmean_tokens': False, 'pooling_mode_last_token': False, 'include_prompt': True})
    (2): Dense({'in_features': 768, 'out_features': 3072, 'bias': False, 'activation_function': 'torch.nn.modules.linear.Identity'})
    (3): Dense({'in_features': 3072, 'out_features': 768, 'bias': False, 'activation_function': 'torch.nn.modules.linear.Identity'})
    (4): Normalize()
)
```

Total number of parameters in the model: 307581696

Generating Embedding

An embedding is a numerical representation of text, like a word or sentence, that captures its semantic meaning. Essentially, it's a list of numbers (a vector) that allows computers to understand the relationships and context of words.

Let's see how EmbeddingGemma would process three different words `["apple", "banana", "car"]`.

EmbeddingGemma has been trained on vast amounts of text and has learned the relationships between words and concepts.

```
In [ ]: words = ["apple", "banana", "car"]

# Calculate embeddings by calling model.encode()
embeddings = model.encode(words)

print(embeddings)
for idx, embedding in enumerate(embeddings):
    print(f"Embedding {idx+1} (shape): {embedding.shape}")

[[-0.18476306  0.00167681  0.03773484 ... -0.07996225 -0.02348064
  0.00976741]
 [-0.21189530 -0.02657359  0.02513712 ... -0.08042689 -0.01999852
  0.00512146]
 [-0.18924113 -0.02551468  0.04486253 ... -0.06377774 -0.03699806
  0.03937357]]
Embedding 1: (768,)
Embedding 2: (768,)
Embedding 3: (768,)
```

The model outputs a numerical vector for each sentence. The actual vectors are very long (768), but for simplicity, those are presented with a few dimensions.

The key isn't the individual numbers themselves, but **the distance between the vectors**. If we were to plot these vectors in a multi-dimensional space, the vectors for `apple` and `banana` would be very close to each other. And the vector for `car` would be far away from the other two.

Determining Similarity

In this section, we use embeddings to determine how semantically similar different sentences are. Here we show examples with high, medium, and low similarity scores.

- High Similarity:
 - Sentence A: "The chef prepared a delicious meal for the guests."
 - Sentence B: "A tasty dinner was cooked by the chef for the visitors."
 - Reasoning: Both sentences describe the same event using different words and grammatical structures (active vs. passive voice). They convey the same core meaning.
- Medium Similarity:
 - Sentence A: "She is an expert in machine learning."
 - Sentence B: "He has a deep interest in artificial intelligence."
 - Reasoning: The sentences are related as machine learning is a subfield of artificial intelligence. However, they talk about different people with different levels of engagement (expert vs. interest).
- Low Similarity:
 - Sentence A: "The weather in Tokyo is sunny today."
 - Sentence B: "I need to buy groceries for the week."
 - Reasoning: The two sentences are on completely unrelated topics and share no semantic overlap.

```
In [ ]: # The sentences to encode
sentence_high = [
    "The chef prepared a delicious meal for the guests.",
    "A tasty dinner was cooked by the chef for the visitors."
]
sentence_medium = [
    "She is an expert in machine learning.",
    "He has a deep interest in artificial intelligence."
]
sentence_low = [
    "The weather in Tokyo is sunny today.",
    "I need to buy groceries for the week."
]

for sentence in [sentence_high, sentence_medium, sentence_low]:
    print(f"👉")
    print(sentence)
    embeddings = model.encode(sentence)
    similarities = model.similarity(embeddings[0], embeddings[1])
    print(f"👉👉 score: ", similarities.numpy()[0][0])

👉
['The chef prepared a delicious meal for the guests.', 'A tasty dinner was cooked by the chef for the visitors.']
👉👉👉 score: 0.8002148
👉
['She is an expert in machine learning.', 'He has a deep interest in artificial intelligence.']
👉👉👉 score: 0.45417833
👉
['The weather in Tokyo is sunny today.', 'I need to buy groceries for the week.']
👉👉👉 score: 0.22262995
```

Using Prompts with EmbeddingGemma

To generate the best embeddings with EmbeddingGemma, you should add an "instructional prompt" or "task" to the beginning of your input text. These prompts optimize the embeddings for specific tasks, such as document retrieval or question answering, and help the model distinguish between different input types, like a search query versus a document.

How to Apply Prompts

You can apply a prompt during inference in three ways.

- Using the `prompt` argument**
Pass the full prompt string directly to the `encode` method. This gives you precise control.
`embeddings = model.encode(sentence, prompt="task: sentence similarity | query: "`
`)`
- Using the `prompt_name` argument**
Select a predefined prompt by its name. These prompts are loaded from the model's configuration or during its initialization.
`embeddings = model.encode(sentence, prompt_name="STS")`
- Using the Default Prompt**
If you don't specify either `prompt` or `prompt_name`, the system will automatically use the prompt set as `default_prompt_name`, if no default is set, then no prompt is applied.
`embeddings = model.encode(sentence)`

```
In [34]: print("Available tasks:")
for name, prefix in model.prompts.items():
    print(f" {name}: \'{prefix}\'"")
print("\n->80")

for sentence in [sentence_high, sentence_medium, sentence_low]:
    print(f"👉")
    print(sentence)
    embeddings = model.encode(sentence, prompt_name="STS")
    similarities = model.similarity(embeddings[0], embeddings[1])
    print(f"👉👉👉 score: ", similarities.numpy()[0][0])
```

Available tasks:
query: "task: search result | query: "
document: "title: none | text: "
BitextMining: "task: search result | query: "
Clustering: "task: clustering | query: "
Classification: "task: classification | query: "
InstructionRetrieval: "task: code retrieval | query: "
MultilabelClassification: "task: classification | query: "
PairClassification: "task: sentence similarity | query: "
Reranking: "task: search result | query: "
Retrieval: "task: search result | query: "
Retrieval-query: "task: search result | query: "
Retrieval-document: "title: none | text: "
STS: "task: sentence similarity | query: "
Summarization: "task: summarization | query: "

```
👉
['The chef prepared a delicious meal for the guests.', 'A tasty dinner was cooked by the chef for the visitors.']
👉👉👉 score: 0.9363755
👉
['She is an expert in machine learning.', 'He has a deep interest in artificial intelligence.']
👉👉👉 score: 0.6425841
👉
['The weather in Tokyo is sunny today.', 'I need to buy groceries for the week.']
👉👉👉 score: 0.38587483
```

Use Case: Retrieval-Augmented Generation (RAG)

For RAG systems, use the following `prompt_name` values to create specialized embeddings for your queries and documents:

- For Queries:** Use `prompt_name="Retrieval-query"`.
`query_embedding = model.encode("How do I use prompts with this model?", prompt_name="Retrieval-query")`
- For Documents:** Use `prompt_name="Retrieval-document"`. To further improve document embeddings, you can also include a title by using the `prompt` argument directly:
 - With a title:**
`doc_embedding = model.encode("The document text...", prompt="title: Using Prompts in RAG | text: ")`
 - Without a title:**
`doc_embedding = model.encode("The document text...", prompt="title: none | text: ")`

Further Reading

- For details on all available EmbeddingGemma prompts, see the [model card](#).
- For general information on prompt templates, see the [Sentence Transformer documentation](#).
- For a demo of RAG, see the [Simple RAG example](#) in the Gemma Cookbook.

Classification

Classification is the task of assigning a piece of text to one or more predefined categories or labels. It's one of the most fundamental tasks in Natural Language Processing (NLP).

A practical application of text classification is customer support ticket routing. This process automatically directs customer queries to the correct department, saving time and reducing manual work.

```
In [ ]: labels = ["Billing Issue", "Technical Support", "Sales Inquiry"]

sentence = [
    "Excuse me, the app freezes on the login screen. It won't work even when I try to reset my password.",
    "I would like to inquire about your enterprise plan pricing and features for a team of 50 people.",
]

# Calculate embeddings by calling model.encode()
label_embeddings = model.encode(labels, prompt_name="Classification")
embeddings = model.encode(sentence, prompt_name="Classification")

# Calculate the embedding similarities
similarities = model.similarity(embeddings, label_embeddings)
print(similarities)

idx = similarities.argmax(1)
print(idx)

for example in sentence:
    print(f"👉", example, "👉👉👉", labels[idx[sentence.index(example)]])

tensor([[0.4673, 0.5145, 0.3604],
        [0.4191, 0.5010, 0.5966]])
tensor([1, 2])
👉 Excuse me, the app freezes on the login screen. It won't work even when I try to reset my password. 👉👉👉 Technical Support
👉 I would like to inquire about your enterprise plan pricing and features for a team of 50 people. 👉👉👉 Sales Inquiry
```

Matryoshka Representation Learning (MRL)

EmbeddingGemma leverages MRL to provide multiple embedding sizes from one model. It's a clever training method that creates a single, high-quality embedding where the most important information is concentrated at the beginning of the vector.

This means you can get a smaller but still very useful embedding by simply taking the first `N` dimensions of the full embedding. Using smaller, truncated embeddings is significantly cheaper to store and faster to process, but this efficiency comes at the cost of potential lower quality of embeddings. MRL gives you the power to choose the optimal balance between this speed and accuracy for your application's specific needs.

Let's use three words `["apple", "banana", "car"]` and create simplified embeddings to see how MRL works.

```
In [ ]: def check_word_similarities():
    # Calculate the embedding similarities
    print("similarity function: ", model.similarity_fn_name)
    similarities = model.similarity(embeddings[0], embeddings[1])
    print(similarities)

    for idx, word in enumerate(words[1:]):
        print(f"👉 apple vs.", word, "👉👉👉 score: ", similarities.numpy()[0][idx])

    # Calculate embeddings by calling model.encode()
    embeddings = model.encode(words, prompt_name="STS")

    check_word_similarities()

similarity function: cosine
tensor([[0.7674, 0.7041]])
👉 apple vs. banana ->👉👉👉 score: 0.767427
👉 apple vs. car ->👉👉👉 score: 0.7040599
```

Now, for a faster application, you don't need a new model. Simply **truncate** the full embeddings to the first **512 dimensions**. For optimal results, it is also recommended to set `normalize_embeddings=True`, which scales the vectors to a unit length of 1.

```
In [ ]: embeddings = model.encode(words, truncate_dim=512, normalize_embeddings=True)

for idx, embedding in enumerate(embeddings):
    print(f"Embedding {idx+1}: {embedding.shape}")

print("\n->80")
check_word_similarities()

Embedding 1: (512,)
Embedding 2: (512,)
Embedding 3: (512,)
```

similarity function: cosine
tensor([[0.7674, 0.7041]])
👉 apple vs. banana ->👉👉👉 score: 0.767427
👉 apple vs. car ->👉👉👉 score: 0.7040599

In extremely constrained environments, you can further shorten the embeddings to just **256 dimensions**. You can also use the more efficient **dot-product** for similarity calculations instead of the standard **cosine** similarity.

```
In [ ]: model = SentenceTransformer(model_id, truncate_dim=256, similarity_fn_name="dot").to(device=device)
embeddings = model.encode(words, prompt_name="STS", normalize_embeddings=True)

for idx, embedding in enumerate(embeddings):
    print(f"Embedding {idx+1}: {embedding.shape}")

print("\n->80")
check_word_similarities()

Embedding 1: (256,)
Embedding 2: (256,)
Embedding 3: (256,)
```

similarity function: dot
tensor([[0.7855, 0.7382]])
👉 apple vs. banana ->👉👉👉 score: 0.7854644
👉 apple vs. car ->👉👉👉 score: 0.7382126

Summary and next steps

You are now equipped to generate high-quality text embeddings using EmbeddingGemma and the Sentence Transformers library. Apply these skills to build powerful features like semantic similarity, text classification, and Retrieval-Augmented Generation (RAG) systems, and continue exploring what's possible with Gemma models.